

ユニット 5・第7週

パイプとプロセスの合成

📅 第7週・日程は未定 📄 演習 TD4（解答 + 生成AIチャレンジ） 🎞 アニメーション 2 本

演習 TD4・解答付き

名前のないパイプとプロセスの組み合わせ

シェルが `ls | grep | wc` をどう組むか：| ごとにパイプ1本、コマンドごとにプロセス1つ、`dup2` で前の出力を次の入力につなぎ、何より使わない端を早めに閉じる。

📝 課題：実習レポート

この実習は短い LaTeX レポート（必要なことだけ）で提出します。扱う内容、テンプレート、例、Prism のガイドは「課題」タブにあります。生成 AI は申告すれば利用できます。

演習問題

- | | |
|--------------------------|-----------------------|
| 1 親子の間のパイプ | 4 固定のコマンドパイプライン |
| 2 パイプの規則：容量、ブロック、SIGPIPE | 5 コマンドラインから受け取るパイプライン |
| 3 大文字の <code>ps</code> | |

TD4 ・ 1

● 解答済み・テスト済み

親子の間のパイプ

課題。 `pipe(2)` でパイプを作り、プロセスを複製してください。親は「Hello World」を 10 回書き、標準エラーに去ることを告げて終了します。子は1回の読み込みで最大 500 バイトを読み、それを表示し、そのことを告げて終了します。定数にはマクロを使いましょう。

解答

C tds/td4/mypipe.c

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a
 * child (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times
 * (default 10)
```

```

* The child reads AT MOST 500 bytes, once, prints them and leaves.
*
* Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the
pipe capacity)
* the parent blocks on write() once the pipe is full. When the child exits,
nobody can read
* any more: the next write() raises SIGPIPE and kills the parent ("Broken
pipe", status 141
* in the shell). That only works if the parent CLOSED its own read end: with
* -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a
potential reader,
* the kernel never raises SIGPIPE and the parent blocks forever.
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE  "Hello World\n"
#define READ_MAX 500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end,
    fds[1] write end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                            /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

```

```
/* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");           /* THE line that
exercise 2 is about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n",
count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave
```

`fds[0]` が読み込み端、`fds[1]` が書き込み端（0 = 入力、1 = 出力と同じ）。`fork` の後は両方のプロセスが両端を持つので、それぞれ使わない端を閉じる。

TD4 ・ 2

● 解答済み・実際の出力

パイプの規則：容量、ブロック、SIGPIPE

課題。親が書く回数を引数で渡してください：50、100、500、1000、5000、10000...。
プログラムが固まったように見えたら、プロセスの状態を確認、ディスクリプタ表を描いて直しましょう。直したプログラムの終了ステータスはいくつですか？

解答

実際の出力

```
$ python3 -c 'import os, fcntl; r, w = os.pipe(); print(fcntl.fcntl(w,
fcntl.F_GETPIPE_SZ), "bytes")'
65536 bytes
```

実際の出力

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 =
killed by timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

実際の出力

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"
I am the child, I read 500 bytes and I leave
exit status of mypipe: 141
```

新規：競合を測る。元の課題文は、親が死ぬのは「何メッセージから」かを見つけるよう求めていました。マルチコアのマシンには決まったしきい値はありません。親がまだ書いている間に子は500バイト読んで去り、どちらが勝つかはスケジューラ次第です。サイズごとに30回実行：

実際の出力

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses;
done
50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
100 messages: 30 x status 0     0 x status 141 (SIGPIPE)
200 messages: 28 x status 0     2 x status 141 (SIGPIPE)
300 messages: 19 x status 0    11 x status 141 (SIGPIPE)
400 messages:  5 x status 0    25 x status 141 (SIGPIPE)
500 messages:  5 x status 0    25 x status 141 (SIGPIPE)
1000 messages: 0 x status 0    30 x status 141 (SIGPIPE)
```

アニメーションで見る → A5.2

1. パイプの容量には限りがあります：Linuxの既定で **65,536 バイト**。10,000 × 12 バイト = 120,000 バイトは入りきらず、満杯になると `write` は誰かが読むまで親を **ブロック** します。

2. 子は 500 バイト読んで終了します。親が自分の読み込み端を閉じていないと (`mypipe_bug`)、カーネルはまだ読み手 (親自身) がいると見なし、親を永遠にブロックしたままにします。3 秒後に `timeout` が親を終了させます (ステータス 124)。
3. 端を閉じていれば、子が去ると読み手はいなくなり、次の `write` が **SIGPIPE** シグナルを受けます。その既定の動作はプロセスの終了です。シェルは「Broken pipe」とステータス $141 = 128 + 13$ を表示します。

アニメーション A5.2・パイプの容量 →

TD4 ・ 3

● 解答済み・テスト済み

大文字の `ps`

課題。子はパイプから読み続け、読んだものを大文字で表示します。親は `dup2` で標準出力をパイプにリダイレクトし、`execlp` で `ps` を実行します。

解答

C tds/td4/upper.c

```
/*
 * TD4 - exercise 3: ps | uppercase, built by hand.
 * The child reads the pipe until end of file and prints it in uppercase.
 * The parent redirects its stdout to the pipe (dup2) and becomes `ps`
 * (execlp).
 * The child sees end of file when `ps` exits, because it was the last
 * writer.
 */
#include <ctype.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define BUFFER_SIZE 512

static void to_upper(int fd_in, int fd_out)
{
    char buffer[BUFFER_SIZE];
    ssize_t n;
```

```

    while ((n = read(fd_in, buffer, sizeof buffer)) > 0) {      /* until end
of file */
        for (ssize_t i = 0; i < n; i++)
            buffer[i] = (char)toupper((unsigned char)buffer[i]);
        exit_if(write_all(fd_out, buffer, (size_t)n) == -1, "write");
    }
    exit_if(n == -1, "read");
}

int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                                             /* child */
        exit_if(close(fds[1]) == -1, "close");                 /* otherwise it never sees
end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }

    /* parent */
    exit_if(close(fds[0]) == -1, "close");
    exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
    exit_if(close(fds[1]) == -1, "close");                     /* 1 already refers to the
pipe */
    execlp("ps", "ps", (char *)NULL);                          /* "ps" twice: file to run,
then argv[0] */
    exit_if(1, "execlp ps");
    return EXIT_FAILURE;
}

```

実際の出力

```

$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?           00:00:25 UVICORN
 9771 ?           00:00:03 SYSTEMD
 9801 ?           00:00:00 (SD-PAM)

```

子は `read` が 0 を返したときに終了しますが、それは書き込み端がすべて閉じたときだけです: `ps` の端 (`ps` が終わるとき) と自分の端。だから子は最初に自分の端を閉じます。 `ps` はパイプに書いていることを知りません。ディスクリプタ 1 がそこを指しているだけです。

TD4 · 4

● 解答済み · テスト済み

固定のコマンドパイプライン

課題。変数 `commands` に定義したパイプライン (たとえば `ls -l | tr -d '[:blank:]'`) を、コマンドがいくつあっても実行できるようにしてください。プログラムは最後のコマンドが終わると終了し、そのステータスを返します。シグナルで死んだ場合はそれを報告し、`EXIT_FAILURE` を返します。

解答

C tds/td4/pipeline.c

```
/*
 * TD4 - exercise 4: run a fixed pipeline, like the shell does for ls -l |
 * tr -d '[:blank:]'
 * Works for any number of commands. Returns the exit status of the LAST
 * command, or
 * EXIT_FAILURE (with a message) if it was killed by a signal.
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

static char *ls[] = { "ls", "-l", NULL };
static char *tr[] = { "tr", "-d", "[:blank:]", NULL };
static char **commands[] = { ls, tr };
static const int ncommands = sizeof commands / sizeof commands[0];

int main(void)
{
    int in = STDIN_FILENO; /* where the next command reads from */
    pid_t last = -1;
```

```

    for (int i = 0; i < ncommands; i++) {
        int fds[2] = { -1, -1 };
        if (i < ncommands - 1)                /* every command but
the last writes to a pipe */
            exit_if(pipe(fds) == -1, "pipe");

        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            if (in != STDIN_FILENO) {          /* not the first: read
from previous pipe */
                exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
                exit_if(close(in) == -1, "close");
            }
            if (fds[1] != -1) {                /* not the last: write
to the new pipe */
                exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
                exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
            }
            execvp(commands[i][0], commands[i]);
            perror(commands[i][0]);
            _exit(127);
        }

        /* parent: close what the child inherited, keep only the read end for
the next one */
        if (in != STDIN_FILENO)
            exit_if(close(in) == -1, "close");
        if (fds[1] != -1) {
            exit_if(close(fds[1]) == -1, "close");
            in = fds[0];
        }
        last = pid;
    }

    /* Wait for every child (no zombies), remember the status of the last
one. */
    int status, last_status = 0;
    pid_t pid;
    while ((pid = wait(&status)) > 0)
        if (pid == last)
            last_status = status;

    if (WIFEXITED(last_status))

```

```
return WEXITSTATUS(last_status);
if (WIFSIGNALED(last_status))
    fprintf(stderr, "pipeline: last command killed by %s\n",
    signame(WTERMSIG(last_status)));
return EXIT_FAILURE;
}
```

実際の出力

```
$ cd demo && ../pipeline
total8
-rw-rw-r--1studentstudent4Sep3013:13one.txt
-rw-rw-r--1studentstudent2Sep3013:13two.txt
```

パターン：各回で次のパイプを作り（最後のコマンドを除く）、子は前のパイプ（**in**）を入力に、新しいパイプを出力にします。親は子がすでに受け継いだものを閉じ、読み込み端を次の回のために残します。最後に親は子を**すべて**待ち（さもないとゾンビが残ります）、最後の子のステータスを保持します。

TD4 ・ 5

● 解答済み・テスト済み

コマンドラインから受け取るパイプライン

課題。前の課題と同じですが、コマンドは **--** で区切ってコマンドラインから受け取ります：
pipeline ls -l -- grep a -- wc -l は **ls -l | grep a | wc -l** と同じです。

解答



C tds/td4/pipeline_args.c

```
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s cmd [args] [-- cmd [args]]...\n",
argv[0]);
        return EXIT_FAILURE;
    }

    /* Split argv in place. commands[k] points to the first word of command
k. */
    char **commands[argc];
    int ncommands = 0;
    commands[ncommands++] = &argv[1];
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            argv[i] = NULL;                /* ends the previous
command */
            if (i + 1 ≥ argc || strcmp(argv[i + 1], "--") == 0 ||
commands[ncommands - 1][0] == NULL) {
                fprintf(stderr, "%s: empty command\n", argv[0]);
                return EXIT_FAILURE;
            }
            commands[ncommands++] = &argv[i + 1];
        }
    }
}
```

実際の出力

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

実際の出力

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

実際の出力

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

コツ： `argv` はもともと `NULL` で終わる文字列の配列です。 `--` をすべて `NULL` に置き換えれば、各コマンドは何もコピーせずに `execvp` にそのまま渡せる `argv` の一部になります。残りは演習 4 と同じです。

チェックリスト

生成AIチャレンジ・あなたのミニシェルにパイプを



目標。 `pipeline_args.c` を再利用してミニシェルに `|` 演算子を加え、わざと起こした行き詰まりを AI と一緒に診断する。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

ミニシェルと、N 個のコマンドのパイプラインを実行するプログラムがあります [両方を貼り付け]。(1) シェルに `|` 演算子を組み込むのを手伝ってください。(2) その後、親の `close()` をわざと 1 つ消します。各プロセスのディスクリプタ表を描きながら、なぜ ``ls | wc -l`` が終わらなくなるのか、そして ``ls -l /proc/<pid>/fd`` でどう確かめるかを説明してください。コードは自分で直さず、質問で私を導いてください。

結果の確かめ方

- ``ls -l | grep c | wc -l`` が `bash` と同じ結果を出す。
- `close()` を消すとコマンドが固まり、``/proc/<pid>/fd`` に書き込み端が開いたまま見える。
- ファイル終端を妨げたディスクリプタをどのプロセスが持っていたか言える。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。